

PAPER2CODE: AUTOMATING CODE GENERATION FROM SCIENTIFIC PAPERS IN MACHINE LEARNING

Minju Seo^{1,3}, Jinheon Baek^{†1}, Seongyun Lee¹, Sung Ju Hwang^{†1,2}

¹KAIST, ²DeepAuto.ai, ³LG AI Research

{minjuseo, jinheon.baek, seongyun, sungju.hwang}@kaist.ac.kr

ABSTRACT

Despite the rapid growth of machine learning research, corresponding code implementations are often unavailable, making it slow and labor-intensive for researchers to reproduce results and build upon prior work. In the meantime, recent Large Language Models (LLMs) excel at understanding scientific documents and generating high-quality code. Inspired by this, we introduce PaperCoder, a multi-agent LLM framework that transforms machine learning papers into operational code repositories. PaperCoder operates in three stages: planning, where it constructs a high-level roadmap, designs the system architecture with diagrams, identifies file dependencies, and generates configuration files; analysis, which focuses on interpreting implementation-specific details; and generation, where modular, dependency-aware code is produced. Moreover, each phase is instantiated through a set of specialized agents designed to collaborate effectively across the pipeline. We then evaluate PaperCoder on generating code implementations from machine learning papers based on both model-based and human evaluations, particularly from the authors of those papers, with author-released repositories as ground truth if available. Our results demonstrate the effectiveness of PaperCoder in creating high-quality, faithful implementations. Furthermore, it consistently shows strengths in the recently released PaperBench benchmark, surpassing strong baselines by substantial margins. Code is available at: <https://github.com/going-doer/Paper2Code>.

1 INTRODUCTION

Reproducibility lies at the heart of scientific progress, which enables researchers to validate findings, build upon prior work, and ultimately push the boundaries of knowledge (Collaboration, 2015; Baker, 2016; Pineau et al., 2021). However, reproducing scientific results remains an enduring challenge. This is often due to incomplete documentation, missing experimental details, lack of access to data or proprietary tools, and, especially in machine learning research, the absence of corresponding code: for example, only average 19.5% of the papers accepted to top-tier machine learning conferences in 2024 provide their code implementations shown in Figure 1. As a result, researchers frequently invest substantial effort in reverse-engineering methods and experimental results from papers, a process that is both time-consuming and labor-intensive, subsequently slowing down the overall pace of science.

Meanwhile, recent Large Language Models (LLMs) have shown outstanding capabilities in understanding and generating both natural language and programming code (Dubey et al., 2024; OpenAI, 2024; Reid et al., 2024), with performances increasingly approaching or even surpassing that of domain experts in some scenarios. In addition, this progress has sparked growing interest in leveraging LLMs to accelerate scientific workflows, particularly in the early stages of ideation for new and valid research hypotheses (Lu et al., 2024; Li et al., 2024; Yang et al., 2024; Si et al., 2024; Yamada et al., 2025; Schmidgall et al., 2025; Baek et al., 2025). Furthermore, some of these studies, as well as others focusing on later stages of automating experimental validations and improvements (Huang et al., 2024; Zhang et al., 2024; Trirat et al., 2024; Chan et al., 2025), demonstrate the potential of LLMs to generate code and even carry out experiments end-to-end; however, they typically assume and heavily rely on access to pre-existing implementations, partial code snippets, or well-defined

[†]Equal Advising.

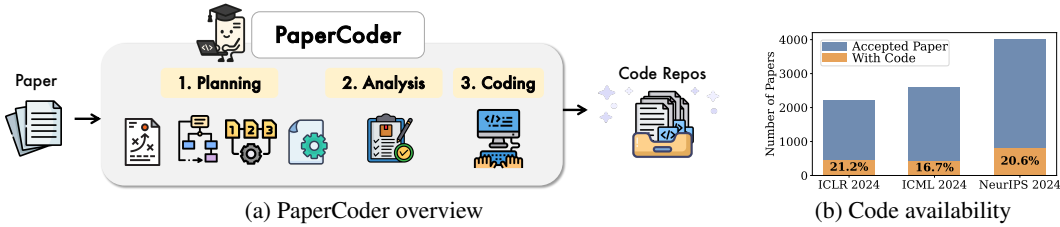


Figure 1: (a) PaperCoder, which aims to transform given scientific papers into code repositories, consisting of planning, analysis, and coding steps. (b) Code availability, where blue bars indicate the total number of accepted papers and orange regions show those with officially released code (See Appendix B.1 for calculation details).

APIs. As such, it remains questionable whether generating faithful implementations solely from papers (without access to prior code, APIs, or additional supplementary materials) can be achievable.

To answer this question, we introduce PaperCoder, a multi-agent LLM-powered framework, designed to automatically generate faithful code repositories in machine learning directly from and contextualized with research papers, which differs from prior work that requires partial implementations from human inputs. Specifically, PaperCoder aims to emulate the typical life cycle of human developers and researchers in writing the repository-level code, by decomposing the task into three structured stages: planning, analysis, and generation. First, during the planning stage, the proposed framework constructs a high-level roadmap to identify core components to implement, draws the overall system architecture with class and sequence diagrams to model structural relationships between modules, identifies file dependencies with their execution orders to guide correct build and execution flows, and generates configuration files to enable flexible customization of experimental workflows by human researchers. This is followed by the analysis stage, performing a fine-grained interpretation of each file and function with respect to their intended functionality, such as required inputs and outputs, interactions with other modules, and any algorithmic or architectural constraints derived from the source paper. Finally, in the generation stage, the framework synthesizes the entire code base based on the execution order determined earlier, along with the artifacts produced in the previous stages.

To validate the effectiveness of PaperCoder, we conduct extensive evaluations on a subset of recent machine learning papers from ICLR, ICML, and NeurIPS referred to as our proposed Paper2Code benchmark (in short, Paper2CodeBench). Also, we incorporate the recent benchmark (Starace et al., 2025) in our evaluation suite, enabling fine-grained evaluations of code implementations. Then, on a battery of tests conducted not only with automated model-based evaluations (covering both reference-free and reference-based settings, conditional on the availability of author-released ground-truth repositories) but also with expert human evaluations (based on authors of original papers), PaperCoder demonstrates substantial improvements over baselines, generating more valid and faithful code repositories that could meaningfully support human researchers in reproducing prior work. Specifically, 88% of the generated repositories by PaperCoder are rated as the best over baselines, and 92% of human judges report that the generated repositories are indeed helpful. Also, analyses show that each component of PaperCoder (consisting of planning, analysis, and generation) contributes to the performance gains, but also that the generated codebases can be executed, sometimes with only minor modifications (averaging 0.81% of total code lines) in cases where execution errors occur.

2 RELATED WORK

Large Language Models for Code LLMs have shown impressive capabilities in text understanding and generation (OpenAI, 2024; Dubey et al., 2024; Reid et al., 2024) and widely utilized for specialized domains (beyond general tasks), such as mathematics, science, and coding (Prabhakar et al., 2025; Wang et al., 2024b; Trinh et al., 2024). Particularly, code-specialized LLMs (Hui et al., 2024; DeepSeek-AI et al., 2024; 2025) have received significant attention thanks to remarkable performance on various software engineering tasks (Xia et al., 2024), including software design and development (Qian et al., 2024; Hong et al., 2024), requirements elicitation (Mu et al., 2023), and formal specification generation (Luo et al., 2024). Our work aligns closely with this line of research, exploring and expanding upon the capabilities and applications of (code-specialized) LLMs.

Repository-Level Coding Early work on code generation typically focuses on single-file tasks, whose objective is to generate short code snippets to solve isolated tasks, such as (algorithmic-level) programming competition problems (Chen et al., 2021; Austin et al., 2021; Hendrycks et al., 2021; Li et al., 2022). However, as LLMs have advanced in comprehending and generating code with the long-context reasoning ability, recent studies have increasingly shifted their attention toward more challenging repository-level coding tasks, which involve generating multi-file repositories that jointly account for architectural design, modular structure, and inter-file dependencies (Liu et al., 2024; Jain et al., 2024; Tang et al., 2024). In particular, several recent efforts explore this emerging paradigm (Zhang et al., 2023; Ouyang et al., 2025), adopting multi-agent or role-based frameworks to emulate realistic development workflows. For instance, ChatDev instantiates LLMs into role-playing agents that collaborate through structured dialogues (Qian et al., 2024), while MetaGPT implements a waterfall-style development pipeline with specialized agents (Hong et al., 2024). Beyond prior work, we explore the underexplored task of transforming full, complex papers into repository-level code.

LLM-Powered Scientific Research LLMs have been adopted to support the scientific process from ideation to experimental validation (Popper, 1959; Qi et al., 2023; Li et al., 2024; Yang et al., 2024; D’Arcy et al., 2024; Liang et al., 2024; Baek et al., 2025; Weng et al., 2025); thereby, helping researchers overcome existing challenges and ultimately accelerate scientific discovery (Lehr et al., 2024; Lu et al., 2024; Yamada et al., 2025). Specifically, in fields such as computer science (where code-based experimentation is central), LLMs have been used to design, refine, and extend code implementations. However, many recent efforts in this space assume access to and build on top of the original codebase (Huang et al., 2024; Trirat et al., 2024; Xiang et al., 2025; Chan et al., 2025), which significantly limits their applicability in real-world scenarios since such implementations are oftentimes unavailable (See Figure 1). To address this, concurrent to our work, Starace et al. (2025) introduces a benchmark dataset called PaperBench, evaluating the capability of existing agentic AI systems in reproducing papers with fine-grained metrics. Notably, on top of PaperBench (which emphasizes evaluation), we further complement and extend this line by focusing on methodological aspects of how to transform scientific papers into repository-level code implementations.

3 METHOD

In this section, we start with describing the task of repository-level code generation from machine learning papers, and propose PaperCoder, a multi-agent, multi-stage framework designed to tackle it.

3.1 REPOSITORY-LEVEL CODE GENERATION FROM MACHINE LEARNING PAPERS

The goal of our repository-level code generation task is to automatically produce a repository that faithfully implements methods and experiments described in machine learning papers (especially for cases where authors do not release their code), to support reproducibility and accelerate scientific progress (Pineau et al., 2021; Magnusson et al., 2023). Formally, we define this task as a function (or a model) M that maps a paper R to a corresponding code repository C , as follows: $M(R) = C$. Here, C is composed of multiple files $\{c_1, c_2, \dots, c_n\}$, each responsible for implementing different components of the methods and experiments in R , but together they should form a cohesive pipeline.

The most straightforward approach to instantiating M is to instruct the LLM to generate the entire code repository, conditioned on the given paper, as follows: $M(R) := \text{LLM}(\mathcal{T}(R))$, where $\mathcal{T}$ is the prompt template that specifies the intended behavior of the LLM for the target task (including task descriptions, detailed instructions, and any other relevant context). Yet, generating a complete, modular, and faithful repository in a single pass is extremely challenging, even for powerful LLMs, due to the inherent complexity of scientific papers and their corresponding implementations, the long-context limitations of current models, and the difficulty in maintaining consistent global structure and cross-file dependencies. Therefore, we propose to decompose the overall task into smaller subtasks, each handled by a specialized agent tailored to a specific aspect of paper-to-code transformation.

3.2 PAPERCODER: LLM-POWERED MULTI-AGENT FRAMEWORK FOR PAPER-TO-CODE

We now introduce PaperCoder, a structured, multi-agent framework for generating code repositories directly from machine learning papers (without access to pre-existing artifacts or implementations,

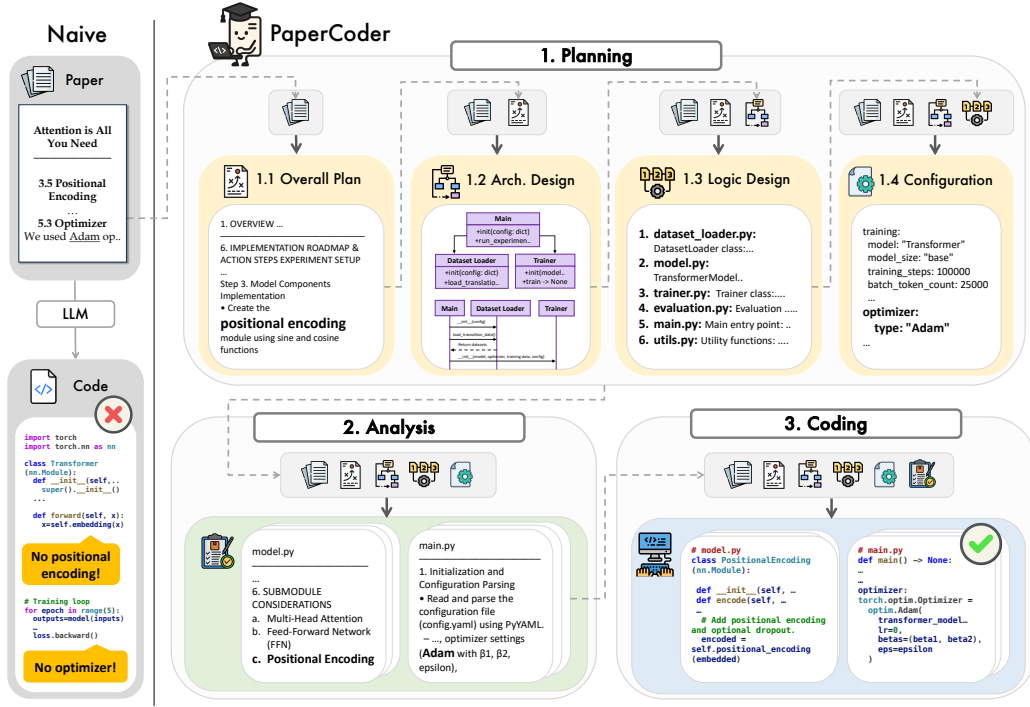


Figure 2: (Left) The naive approach, which directly generates an entire code repository from a paper. (Right) Our PaperCoder framework, which is operationalized by decomposing the task into three stages: (1) Planning, where a high-level implementation plan is constructed from the paper, including overall plan, architectural design, logic design, and configuration file; (2) Analysis, where the plan is translated into detailed file-level specifications; and (3) Coding, where the final codes are generated to implement the methods and experiments of the paper.

such as skeleton code). Specifically, inspired by typical software development workflows, PaperCoder decomposes the task into three coordinated stages: Planning, Analysis, and Coding, each orchestrated by specialized LLM agents. Formally, given a paper R , the overall process can be defined as follows:

Planning: $P = M_{\text{plan}}(R)$, **Analysis:** $A = M_{\text{analysis}}(R, P)$, **Coding:** $C = M_{\text{code}}(R, P, A)$,

where P , A , and C represent the high-level implementation plan, the detailed function-level analysis, and the final code repository, respectively. The overall pipeline of PaperCoder is shown in Figure 2.

3.2.1 PLANNING

It is worth noting that, in contrast to implementation specifications designed explicitly for software development, papers are written to communicate ideas and findings to humans. As a result, they often contain high-level motivations, persuasive narratives, and auxiliary details that are crucial for human understanding but noisy, loosely specified, or ambiguous from a software engineering perspective. To mitigate this, we introduce a planning phase that transforms unstructured textual content into implementation-level abstractions. Also, we decompose the planning process into four sequential subcomponents (to simplify the task and reduce cognitive load of LLM-powered agents at each step): 1) overall plan, 2) architecture design, 3) logic design, and 4) configuration generation. Formally, we define this as: $M_{\text{plan}}(R) \rightarrow P = \{o, d, l, g\}$, where o is the overall plan, d is the architecture design, l is the logic design, and g is the configuration file, with each stage using the outputs of the previous ones as contextual input. We then describe how each subcomponent is instantiated below.

Overall Plan The first step is to extract a high-level summary of the core components and functionalities described throughout the paper, to identify the specific methods and experiments to be implemented. In other words, this high-level overview includes model components, training objectives, data processing steps, and evaluation protocols (distributed across the entire paper), which can form the foundation for all subsequent steps, formalized as follows: $M_{\text{plan}}^{(1)}(R) := \text{LLM}(\mathcal{T}_{\text{plan}}^{(1)}(R)) \rightarrow o$.

Architecture Design Based on the extracted overall plan alongside the input paper, the next step is to define the repository-level architecture, which includes identifying files, organizing them into modules, and defining their relationships, to ensure a coherent and maintainable structure. Specifically, the LLM-powered agent is prompted to generate a file list, which outlines the overall file structure of the repository; a class diagram, which details static representations of files (such as core classes and their attributes); and a sequence diagram, which models the dynamic interactions. Formally, similar to overall plan, this process can be defined as follows: $M_{\text{plan}}^{(2)}(R, o) := \text{LLM}(\mathcal{T}_{\text{plan}}^{(2)}(R, o)) \rightarrow d$.

Logic Design While the previous architecture design focuses on what to build, the logic design phase specifies how these components should be instantiated in practice by considering their dependencies in terms of overall execution flow. This step is crucial because individual modules often depend on shared utilities, configurations, or data loaders that are defined in other parts of the repository, and without an explicitly defined execution order, the code generation can result in failure or inconsistency (e.g., generating file B before file A when B imports modules from A). To address this, the logic design stage not only produces an ordered file list that dictates the sequence in which the files should be implemented and executed, but also further elaborates on the logic within each file; thereby, providing more fine-grained specifications. Formally, $M_{\text{plan}}^{(3)}(R, o, d) := \text{LLM}(\mathcal{T}_{\text{plan}}^{(3)}(R, o, d)) \rightarrow l$.

Configuration Generation In the last stage of planning, PaperCoder synthesizes a configuration file (`config.yaml`) that includes key hyperparameters, model settings, and other runtime options based on prior outputs alongside the given paper. We note that, in addition to grounding the code generation process with the explicit configuration details, it enables researchers to easily review and adjust experimental configurations without modifying the source code. Formally, $M_{\text{plan}}^{(4)}(R, o, d, l) := \text{LLM}(\mathcal{T}_{\text{plan}}^{(4)}(R, o, d, l)) \rightarrow g$. We provide prompts used to elicit each planning output in Appendix D.

3.2.2 ANALYSIS

Following the planning stage, which defines the overall structure and execution flow of the repository, the analysis phase focuses on interpreting and specifying the implementation-level details for modules within each file. In other words, unlike planning that answers what components to build and how they relate, this phase addresses the question of how each component should be operationalized and concretely implemented at the file level, which includes the definition of functional goals, input-output behaviors, intra- and inter-file dependencies, and algorithmic specifications derived from the original paper. Specifically, given the input paper R and planning outputs $P = \{o, d, l, g\}$, the analysis agent iteratively processes each file f_i (identified during planning) and generates a detailed analysis a_i describing what needs to be implemented in that file. Formally, $\{M_{\text{analysis}}(R, P, f_i)\}_{i=1}^{n=|F|}$ where $M_{\text{analysis}}(R, P, f_i) := \text{LLM}(\mathcal{T}_{\text{analysis}}(R, P, f_i)) \rightarrow a_i$, with F as the set of identified files, e.g., $f_i \in F$.

3.2.3 CODING

The final stage is the coding phase, where the complete code repository is produced. In particular, each file is generated based on all the available contextual information accumulated from the previous stages, including the overall plan, architecture design, logic design, configuration file, and file-specific analyses, as well as the original paper. Additionally, to ensure consistency across different files, we generate them sequentially according to the execution order (i.e., the ordered file list determined during the logic design stage). To be formal, for each file f_i , the corresponding code c_i is generated as follows: $M_{\text{code}}(R, P, f_i, a_i, \{c_1, \dots, c_{i-1}\}) := \text{LLM}(\mathcal{T}_{\text{code}}(R, P, f_i, a_i, \{c_1, \dots, c_{i-1}\})) \rightarrow c_i$, resulting in the complete code repository $C = \{c_i\}_{i=1}^{n=|F|}$. We note that this iterative formulation can ensure that i -th code is generated with full awareness of its dependencies and the evolving state of the repository.

4 EXPERIMENT

We now describe the experimental setup and the experimental results with reproducibility analyses.

4.1 EXPERIMENTAL SETUP

Datasets To evaluate our PaperCoder, we construct a new benchmark (**Paper2CodeBench**). Specifically, we collect the accepted papers from recent machine learning venues (such as ICLR, ICML, and NeurIPS 2024) with the OpenReview API¹, and filter them based on the availability of code with its total number of tokens less than 70,000, to ensure the full repository remains within reasonable processing limits of modern LLMs for generation and evaluation. Also, to maintain the quality, we perform model-based evaluation (Liu et al., 2023) with GPT-4o on all the collected repositories and select the top 30 from each venue, resulting in a total of 90 papers listed in Tables 20, 21, and 22. Moreover, we additionally consider 21 papers for human evaluation (See Table 23). In addition to Paper2CodeBench, we also use the recently released **PaperBench Code-Dev** (Starace et al., 2025), which consists of 20 papers from ICML 2024 with paper-specific rubrics annotated by humans. In particular, those rubrics are used to judge the correct implementation based on LLM-based evaluation.

Baselines and Our Model We target the novel problem of Paper2Code, and there are no baselines designed for it to enable direct comparison. Nevertheless, we consider several related approaches proposed to implement repository-level code (or the entire software) from natural language inputs (such as software requirements), in addition to the ablated variants of our full PaperCoder framework, as follows: **ChatDev** (Qian et al., 2024) is a multi-agent framework for software development, where several role-specific LLM-powered agents collaborate via structured dialogues; **MetaGPT** (Hong et al., 2024) similarly adopts a role-based multi-agent paradigm, but its process is organized by the principle of Standardized Operating Procedures (SOPs); **Abstract** is a variant of our PaperCoder, which uses only the paper abstract for implementation; **Paper**, while using the full paper, performs one-shot code generation; **PaperCoder (Ours)** is our full framework, structured into three stages of planning, analysis, and code generation. Additionally, for the PaperBench Code-Dev, we consider baselines suggested by it: **Basic Agent** is the agentic architecture that can run a predefined set of tools with the ReAct-style approach (Yao et al., 2023), built upon the agent from Inspect AI², and **Iterative Agent** that extends Basic Agent, iteratively instructing the model to complete the next subtask.

Evaluation Setup Recall that, as shown in Figure 1, the official code implementations of many papers are not available; however, manually annotating their corresponding code implementations to evaluate the quality of automatically generated code repositories is highly labor-intensive and challenging. To address this and ultimately perform the evaluation at scale, we design two evaluation protocols: reference-based (when ground-truth code is available) and reference-free (when it is not), following the recent trends in using LLMs as a judge (Zheng et al., 2023; Fu et al., 2024; Liu et al., 2023). In addition to this, we also perform human evaluations with the authors of the original papers, to ensure reliable judgments and to assess the quality of our model-based evaluations by measuring their correlation with human scores. We discuss each evaluation protocol in detail below.

- *Reference-Based Evaluation.* We use the official author-released repository as the gold standard only if it is available, since it most accurately reflects the implementations intended by the authors, including the components they consider essential to their main ideas. Specifically, we prompt the model (such as o3-mini-high³) to judge the quality of the generated repository with respect to the gold repository, alongside the input paper as context (See Appendix D for the detailed prompt). The model then identifies components (to be implemented), categorizes them into three severity levels (high, medium, and low), and critiques how well each component is implemented. After that, it returns the overall score on a 5-point Likert scale. We note that, to ensure the reliability of the model-based evaluation, we sample multiple outputs (e.g., 8) and report the average score.
- *Reference-Free Evaluation.* For cases where the official author-released code is not available, we introduce the reference-free evaluation protocol that leverages only the paper to assess the quality of its generated repository. Similar to the reference-based evaluation, the evaluation model is prompted to identify key components, categorize them by severity, and critique their implementations in the generated code, but they are performed solely based on the information provided in the paper. The rest of the evaluation process, such as sampling and score averaging, follows the same setup.

¹<https://docs.openreview.net/reference/api-v2>

²<https://inspect.ai-safety-institute.org.uk/agents.html#sec-basic-agent>

³Unless otherwise stated, we use o3-mini-high due to strong code understanding and reasoning capability.

Table 1: Results on our Paper2CodeBench, where we report average scores and standard deviations (in parentheses) grouped by conferences. Oracle denotes the evaluation results with the official repository released by the paper authors. Also, on the right side, we report statistics on the number of tokens, files, and functions, averaged over all implementations. Bold indicates the best scores, statistically significant than baselines ($p \leq 0.05$).

	Reference-Based Evaluation			Reference-Free Evaluation			Statistics		
	ICLR	ICML	NeurIPS	ICLR	ICML	NeurIPS	# of Tokens	# of Files	# of Funcs
ChatDEV	2.70 (0.63)	2.97 (0.58)	2.96 (0.69)	4.00 (0.65)	4.12 (0.53)	4.01 (0.74)	6150.54	6.99	23.82
MetaGPT	2.48 (0.48)	2.75 (0.70)	2.95 (0.87)	3.52 (0.60)	3.63 (0.75)	3.59 (0.92)	5405.21	3.24	18.08
Abstract	2.28 (0.42)	2.43 (0.49)	2.35 (0.62)	3.03 (0.64)	3.01 (0.60)	2.99 (0.78)	3376.99	1.28	12.62
Paper	3.08 (0.66)	3.28 (0.67)	3.22 (0.80)	4.15 (0.63)	4.30 (0.53)	4.08 (0.84)	3846.33	1.79	14.84
PaperCoder	3.68 (0.52)	3.72 (0.54)	3.83 (0.50)	4.73 (0.32)	4.73 (0.44)	4.77 (0.38)	14343.38	6.97	35.22
Oracle	N/A	N/A	N/A	4.84 (0.26)	4.80 (0.32)	4.83 (0.38)	32149.04	28.00	122.03

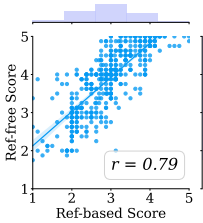


Figure 3: Correlation between model-based evaluations: reference-based and reference-free.

Table 2: Results with human evaluation. For model-based evaluations (both reference-based and reference-free), 5-point Likert evaluation scores are converted to rankings for comparability with human ranking results. Human rankings are also converted to scores of 5 (top repository), 3 (middle repository), and 1 (bottom repository).

	Score ($\uparrow$)			Ranking ($\downarrow$)		
	Ref-based	Ref-free	Human	Ref-based	Ref-free	Human
Abstract	2.26 (0.37)	2.94 (0.61)	2.68 (0.56)	2.96 (0.20)	2.96 (0.00)	2.70 (0.56)
Paper	3.00 (0.54)	3.91 (0.63)	2.76 (1.20)	1.92 (0.41)	1.88 (0.38)	2.09 (0.60)
PaperCoder (Ours)	3.66 (0.43)	4.55 (0.51)	4.60 (1.00)	1.08 (0.28)	1.08 (0.28)	1.22 (0.52)
ChatDEV	2.68 (0.60)	3.82 (0.37)	2.12 (1.17)	2.58 (0.50)	2.23 (0.59)	2.43 (0.59)
MetaGPT	2.61 (0.54)	3.39 (0.67)	2.12 (1.17)	2.38 (0.58)	2.46 (0.51)	2.43 (0.59)
PaperCoder (Ours)	3.66 (0.43)	4.55 (0.51)	4.76 (0.88)	1.04 (0.20)	1.04 (0.20)	1.13 (0.46)

- *Human Evaluation.* While model-based evaluation offers a scalable and automated way of assessment, we also conduct human evaluations to validate our PaperCoder based on expert-grounded evaluation. Specifically, to ensure informed and accurate judgment, each participant is assigned a paper for which they are the first author. Also, they are presented with multiple implementations generated by different approaches, and asked to rank them. We offer more details in Appendix A.2.

Lastly, for evaluation on the PaperBench Code-Dev benchmark (Starace et al., 2025), we follow their evaluation setup, measuring the score over the paper-specific rubrics with LLM-based evaluation.

4.2 EXPERIMENTAL RESULTS AND ANALYSIS

Main Results Table 1 presents main results on Paper2CodeBench, in which PaperCoder consistently outperforms all baselines. We hypothesize that this performance gap stems from its top-down behavior, analyzing full papers thoughtfully before generation, unlike prior approaches that typically follow a bottom-up strategy, which begins with and expands short requiremental descriptions (via role-playing or SOP). In other words, the top-down approach, operationalized through the sequence of planning, analysis, and coding, is effective in handling long-form scientific documents, which are often loosely structured from a software engineering perspective. Also, when compared to the non-comparable Oracle setting (which performs evaluations on the author-released repositories), PaperCoder achieves performance that is on par, without statistically significant differences, demonstrating its effectiveness in faithfully implementing code whose quality is closer to the implementation by authors.

Correlation between Reference-Based and Reference-Free Evaluation Recall that the reference-free evaluation protocol is designed for cases where the ground-truth repository is not available, and to investigate whether it works as a reliable proxy for the reference-based evaluation protocol, we measure their rank correlation on all samples from Paper2CodeBench. Then, as shown in Figure 3, there is a strong positive correlation between them, achieving a Pearson correlation coefficient of $r = 0.79$. This result supports that the reference-free evaluation can serve as a reliable proxy for the reference-based evaluation, ultimately functioning as a standalone metric to assess the code quality.

Table 3: PaperBench Code-Dev results. We report the averaged performance over three runs with standard deviations.

Model	Replication Score (%)	
	o3-mini-high	claude-3.5-sonnet
BasicAgent	5.1 $\pm$ 0.8	35.4 $\pm$ 0.8
IterativeAgent	16.4 $\pm$ 1.4	27.5 $\pm$ 1.6
PaperCoder	45.14 $\pm$ 0.3	51.14 $\pm$ 1.4

Table 4: Results based on both model-based and human evaluations with varying backbone LLMs for PaperCoder.

		DS-Coder	Qwen-Coder	DS-Distill-Qwen	o3-mini-high
Score ($\uparrow$)	Ref-based	1.47 (0.46)	1.78 (0.28)	2.05 (0.25)	3.66 (0.43)
	Ref-free	1.62 (0.54)	2.09 (0.22)	2.31 (0.24)	4.55 (0.51)
	Human	1.32 (0.58)	2.71 (1.12)	3.29 (0.98)	4.68 (0.80)
Ranking ($\downarrow$)	Ref-based	3.46 (0.00)	2.92 (0.88)	2.25 (0.65)	1.00 (0.20)
	Ref-free	3.50 (0.00)	2.88 (0.83)	2.12 (0.54)	1.00 (0.25)
	Human	3.74 (0.45)	2.74 (0.86)	2.30 (0.70)	1.22 (0.60)

Table 6: Ablation results on the subset of Paper2CodeBench with scores and standard deviations.

	Ref-based	Ref-free
Paper	3.28 (0.67)	4.30 (0.53)
+ Overall Plan	3.40 (0.57)	4.34 (0.58)
+ Arch. Design	3.13 (0.68)	4.07 (0.74)
+ Logic Design	3.60 (0.52)	4.50 (0.57)
+ Config File	3.66 (0.45)	4.45 (0.53)
+ Analysis (Ours)	3.72 (0.54)	4.73 (0.44)

Table 7: Results of the PaperCoder and PaperCoder with Self-Refine, under the reference-based evaluation protocol.

	PaperCoder	w/ Self-Refine
Overall Plan	4.67	4.87 (+0.20)
Arch. Design	3.20	3.96 (+0.76)
Logic Design	4.09	4.38 (+0.29)
Config File	2.93	3.93 (+1.00)
Analysis	4.18	4.32 (+0.14)
Code	3.39	3.89 (+0.50)

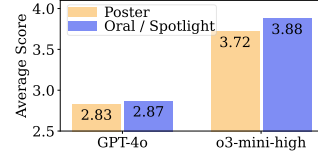


Figure 4: Model-based evaluation results by paper presentation types.

Human Evaluation Results In addition to automatic evaluations, we conduct human evaluations and report the results in Table 2. From this, we confirm that PaperCoder achieves the best ranking, consistent with model-based evaluations, which reaffirms its effectiveness. Also, to ensure whether the model-based evaluations are a reasonable proxy to judge the implementation quality, we measure their correlations with human evaluation scores. As shown in Table 5, we observe strong rank correlations across both reference-based and reference-free settings, which suggests that model-based evaluation can reliably approximate human judgment. Also, based on this result, we use o3-mini-high as the default evaluation model. Lastly, we ensure the quality and reliability of human evaluations by measuring the inter-annotator agreement based on Cohen’s kappa coefficient, which exhibits a high score of 0.79, indicating strong consistency.

Results on PaperBench Code-Dev In addition to our Paper2CodeBench, we further validate the effectiveness of PaperCoder on another PaperBench Code-Dev dataset, which enables fine-grained evaluations for code implementations. As Table 3 shows, PaperCoder achieves the highest replication scores across two different LLMs of o3-mini-high and Claude 3.5 Sonnet, substantially outperforming baselines designed for PaperBench Code-Dev. These results further demonstrate the generalizability and robustness of PaperCoder across diverse evaluation benchmarks and models.

Analysis on Different LLMs Extending the model variations results on PaperBench Code-Dev, we conduct an auxiliary analysis with DS-Coder (DeepSeek-Coder-V2-Lite-Instruct; DeepSeek-AI et al., 2024), Qwen-Coder (Qwen2.5-Coder-7B-Instruct; Hui et al., 2024), DS-Distill-Qwen (DeepSeek-R1-Distill-Qwen-14B; DeepSeek-AI et al., 2025), and o3-mini-high (the high reasoning-effort variant of o3-mini) on Paper2CodeBench. As summarized in Table 4, the proprietary model (o3-mini-high) consistently outperforms all other backbones across all evaluation settings. Among other open-source models, DS-Distill-Qwen performs the best, followed by Qwen-Coder and DS-Coder. These results suggest the importance of selecting a capable backbone to instantiate PaperCoder, particularly one with strong reasoning capabilities. Also, based on this, we primarily use o3-mini-high as the basis.

Ablation Studies To see how much each component of PaperCoder contributes to the performance gain, we conduct ablation studies on the subset of Paper2CodeBench (composed of ICML papers). Specifically, we start with the method that uses only the full paper and incrementally add components in the order they are executed (such as overall plan, architecture design, logic design, configuration generation, and analysis), reported in Table 6. From this, we observe that the performance steadily improves as additional components are incorporated. Meanwhile, a performance drop occurs when the architecture design module is added; however, while this might seem surprising at first, it is in fact expected: architecture design alone does not specify the execution or implementation order of files, which leads to confusion during the code generation (see Figure 7). However, this issue is addressed once the subsequent logic design module explicitly defines file dependencies and establishes a clear

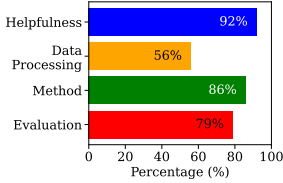


Figure 5: Fine-grained analyses on code by PaperCoder.

Table 8: Replication scores on 10 papers from PaperBench, including execution and result match.

Model	Score (%)
BasicAgent	2.60
IterativeAgent	11.22
PaperCoder	28.46

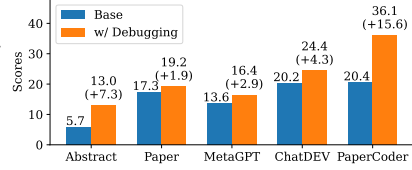


Figure 6: Results on the author-written rubric for papers from Paper2CodeBench (human evaluated), with gains in parentheses.

generation order. Overall, integrating all modules yields the highest performance, confirming the effectiveness of our fully structured, multi-stage pipeline with various modules proposed.

Experiment with Refinement We confirm in Table 6 that the planning and analysis stages play a pivotal role in guiding subsequent analysis and coding, and we further test whether refining earlier outputs can improve downstream performance. Specifically, we augment the planning and analysis phases with verification-and-refinement steps (See Figures 23 to 32 for prompts), following Self-Refine (Madaan et al., 2023), and evaluate a total of 30 papers subsampled from Paper2CodeBench (10 from each conference). As shown in Table 7, refinement of planning and analysis improves their own outputs but also leads to measurable gains in the subsequent stages, reducing downstream errors.

Correlation on Paper Type To see whether the acceptance category (or presentation format) of papers correlates with the quality of their corresponding implementations by PaperCoder, we analyze it by separating papers into oral/spotlight and poster categories on Paper2CodeBench (which includes 14 oral or spotlight papers and 76 poster papers). As shown in Figure 4, scores are slightly higher for oral/spotlight papers on model-based evaluations with GPT-4o and o3-mini, suggesting that papers with higher recognition might reflect clearer writing, probably leading to faithful code generation. For further analysis on how the completeness of papers impacts the results, please refer to Table 12.

Fine-Grained Analysis of Generated Repositories To more thoroughly evaluate the quality and practical utility of the generated code, we conduct a set of fine-grained human analyses according to its usability for reproduction and its component-wise implementation quality. Specifically, we ask annotators whether the top-ranked repository from PaperCoder would make reproducing the original work easier than starting from scratch, and 92% agree, highlighting its practical value. Also, we conduct a component-level analysis to assess which parts of the papers are most effectively translated into code, by asking human annotators to identify key elements for Data Processing, Method, and Evaluation, then measure how many are actually implemented. As shown in Figure 5, the coverage reaches 80% for Method and 79% for Evaluation. Notably, among the errors observed, many of them originate from the Data Processing stage, where papers often under-specify details about data formats, preprocessing steps, or loading procedures. Lastly, to investigate why human annotators prefer PaperCoder over its baselines and ablated variants (with 22 out of 25 selecting the repositories from PaperCoder), we ask them to provide the reasons for their choices, and the majority of which are completeness, clean structure, and faithfulness to the original papers, summarized in Table 16.

4.3 ADDITIONAL ANALYSIS ON REPRODUCTION FROM IMPLEMENTED CODE REPOSITORY

While our focus is on generating faithful implementations that can aid research, we further examine whether these implementations can fully reproduce the original experimental results end-to-end.

Analysis on Executability It is worth noting that making the repository-level code executable and fully reproducible in one go is extremely challenging (even for humans), as demonstrated by Starace et al. (2025). Also, our goal is to provide a faithful starting point that meaningfully aids reproduction efforts (Figure 5), rather than aiming for perfect reproduction. Nevertheless, to assess how close our generated repositories are to being directly executable, we perform manual execution evaluations on five papers. Specifically, when execution fails, we manually debug and refine the code and adapt the input data as needed to enable successful runs. We then find that, on average, only 0.81% of the code lines require minor modification, such as updating deprecated API or correcting data type mismatches, for successful execution (see examples in Figures 8 to 12 with statistics in Table 15), which highlights that our generated repositories are near-executable with minimal human intervention.

Analysis on Reproducibility An equally important, though not our primary focus, question is whether the generated repositories can reproduce the results intended by the original authors. To examine this, we sample 10 papers from PaperBench and another 10 from the human evaluation set of Paper2CodeBench. Also, we automatically invoke LLM-assisted debugging (only when execution errors occur), where the model was provided with error messages, source code, and relevant training data (if needed) to resolve issues. First, for PaperBench, we use the full rubric provided, including the aspects of result match as well as code development and execution, with o3-mini serving as the judge. Then, as shown in Table 8, PaperCoder achieves the highest score. Also, for Paper2CodeBench, we adopt the rubric defined by the paper authors, covering Data Processing, Method, and Evaluation, with o4-mini as the judge, and as shown in Figure 6, PaperCoder outperforms all baselines regardless of whether debugging is used. These results show that its repositories are not only executable with minimal (and automatically debuggable) intervention but also more faithfully reproduce the papers.

Case Study We further conduct a manual case study on five repositories, where annotators check whether the returned outputs match the reported results. As described in Table 18 with Appendix A.5, four reproduce results (at least partially), while one fails due to issues in loss function design.

5 CONCLUSION

In this work, we introduced PaperCoder, a framework that automatically generates code repositories from research papers in machine learning through a structured, three-stage pipeline. Specifically, we defined a high-level roadmap, system architecture, execution logic, and configuration via the planning stage, which are then enhanced through detailed per-file analysis, followed by the sequential code generation informed by artifacts from prior stages. To validate PaperCoder, we performed evaluations on two benchmarks: our Paper2CodeBench, comprising recent papers from top-tier machine learning venues, and (recently released) PaperBench Code-Dev, providing fine-grained evaluation protocols, on which PaperCoder consistently outperforms existing baselines on both model-based and human evaluations. Furthermore, additional analyses demonstrate its robustness and practicality: it remains effective across different LLM backbones, shows practical executability with only 0.81% of the lines requiring minor fixes, and benefits from each stage in the pipeline. We envision PaperCoder as one important step toward accelerating scientific progress by aiding the reproduction of research papers.

ETHICS STATEMENT

Our work aims to generate faithful code repositories from scientific papers in machine learning, and we believe it has a substantial positive impact in contributing to open science and facilitating rapid experimentation. However, we also acknowledge potential risks and misuse of our framework. For example, some papers intentionally refrain from releasing implementations due to security concerns, such as those involving jailbreaking or exploitation techniques. Yet, our method could potentially be used to reproduce such sensitive implementations. To address such risks, in real-world production, it would be necessary to develop and incorporate safeguards (such as harmful content filters, protective prompting, and secure execution environments) to ensure responsible and safe use of our framework.

REPRODUCIBILITY STATEMENT

We provide the public GitHub link to the code to reproduce our work. Detailed instructions for running the experiments are included in the accompanying README files, and furthermore, all necessary details to reproduce our experiments are described in Section 4.1 and in Appendix A.1.

ACKNOWLEDGEMENTS

This work was supported by the Institute for Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (RS-2019-II190075, Artificial Intelligence Graduate School Program (KAIST)), the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2023-00256259), the Institute of Information & Communications Technology Planning & Evaluation (IITP) with a grant funded by the Ministry of Science and ICT (MSIT) of the Republic of Korea in connection with the Global AI Frontier Lab International Collaborative Research (No. RS-2024-00469482 & RS-2024-00509279), and the InnoCORE program of the Ministry of Science and ICT (No. N10250156).